Министерство образования Республики Беларусь
Учреждение образования
«БЕЛОРУССКИЙ ГОСУДАРСТВЕННЫЙ
УНИВЕРСИТЕТ ИНФОРМАТИКИ И
РАДИОЭЛЕКТРОНИКИ»

Факультет компьютерных систем и сетей

Кафедра электронных вычислительных машин

Дисциплина: Жизненный цикл разработки программного обеспечения

ОТЧЕТ
по лабораторной работе №
2 на тему
«СИСТЕМА УПРАВЛЕНИЯ ВЕРСИЯМИ»

Студент гр. 434701: Е.А. Юденко

Преподаватель: О.М. Внук

Минск 2026

СОДЕРЖАНИЕ

1 ЦЕЛЬ И ЗАДАЧИ 3
2 КРАТКИЕ ТЕОРЕТИЧЕСКИЕ СВЕДЕНИЯ 4
2.1 Системы контроля версий 4
2.2 Git и его особенности 4
2.3 Основные приемы работы с Git 4
3 ОПИСАНИЕ ВЫПОЛНЕННОГО ВАРИАНТА ЗАДАНИЯ 5
3.1 Проверка Git 5
3.2 Настройка конфигурации Git 5
3.3 Fork репозитория 6
3.4 Выполнение операций с репозиторием в консоли 7
3.5 Создание ветки ProjectTemplate 8
4 ОТВЕТЫ НА КОНТРОЛЬНЫЕ ВОПРОСЫ 10
4.1 В чем преимущество Git перед SVN? 10
4.2 В чем преимущество распределенных репозиториев? 10
4.3 Граф типичной работы с веткой master в Git 10
4.4 Наиболее часто используемые команды 10
4.5 Что делать при изменениях в удаленном репозитории? 11
ЗАКЛЮЧЕНИЕ 12

1 ЦЕЛЬ И ЗАДАЧИ

Цель работы – изучить порядок работы с Git, освоить основные операции с репозиторием и получить навык совместной работы над проектом.

В соответствии с заданием требуется настроить Git с использованием SSH-авторизации, обеспечивающей безопасное подключение к удаленному репозиторию без ввода пароля при каждой операции. Также необходимо выполнить fork исходного репозитория с проектом, чтобы получить собственную копию для дальнейшей работы. В рамках выполнения задания создаются две ветки – «lab1» и «ProjectTemplate», в которых настраивается файл «.gitignore», выполняются commit и push изменений, а также слияние веток в мастер после проверки преподавателем.

Для выполнения работы поставлены следующие задачи:

1 Установить и настроить Git (имя, группа, SSH-ключ).

2 Сделать fork репозитория и клонировать его локально.

3 Создать ветку «lab1», перенести материалы лабораторной работы №1, настроить «.gitignore», выполнить commit и push.

4 Создать ветку «ProjectTemplate», добавить README.md и пустой проект, настроить «.gitignore», выполнить commit и push.

5 Слить ветку «ProjectTemplate» в мастер после одобрения преподавателя.

6 Проанализировать изменения в репозитории на GitHub и оформить отчет.

2 КРАТКИЕ ТЕОРЕТИЧЕСКИЕ СВЕДЕНИЯ

2.1 Системы контроля версий

Система контроля версий (VCS) – программное обеспечение для отслеживания изменений файлов, сохранения истории и совместной работы над проектом. Основная задача VCS – фиксировать состояние проекта, чтобы можно было вернуться к предыдущим версиям и выявить, кем и когда была внесена модификация.

VCS делятся на централизованные (SVN, CVS) – с единственным центральным репозиторием, и распределенные (Git, Mercurial) – где каждый разработчик имеет полную локальную копию репозитория с историей.

Распределенный подход обеспечивает автономную работу и синхронизацию с другими репозиториями только при необходимости.

2.2 Git и его особенности

Git – распределенная система контроля версий, созданная Линусом Торвальдсом. Git стал стандартом в современной разработке благодаря гибкости, скорости и поддержке ветвления.

Ключевые особенности Git:

– локальность операций: почти все действия (коммит, история, ветки, слияние) выполняются без сетевого подключения;

– снимки, а не различия: Git сохраняет полный снимок файлов на момент коммита, а для не изменившихся ссылку на предыдущую версию;

– легкое ветвление: ветки это указатели на коммиты, поэтому создание и слияние выполняются быстро;

– три состояния файлов: измененный (modified), подготовленный (staged) и зафиксированный (committed);

– область подготовки (staging area): позволяет выбирать, какие изменения войдут в следующий коммит.

2.3 Основные приемы работы с Git

Основной цикл работы с Git включает следующие этапы:

1 Клонирование удаленного репозитория создает локальную копию проекта.

2 Внесение изменений в файлы рабочего дерева.

3 Добавление изменений в область подготовки.

4 Фиксация подготовленных изменений в локальной истории.

5 Отправка изменений в удаленный репозиторий.

Для работы с ветками предусмотрены операции создания, переключения и слияния.

Файл «.gitignore» задает файлы и папки, игнорируемые Git (временные файлы, логи, артефакты сборки).

3 ОПИСАНИЕ ВЫПОЛНЕННОГО ВАРИАНТА ЗАДАНИЯ

3.1 Проверка Git

Перед выполнением работы проверено наличие Git на рабочем компьютере. Для проверки использована команда git --version.

Команда вывела версию установленного Git, что подтверждает готовность системы к выполнению работы. Результат приведён на рисунке 3.1.1.

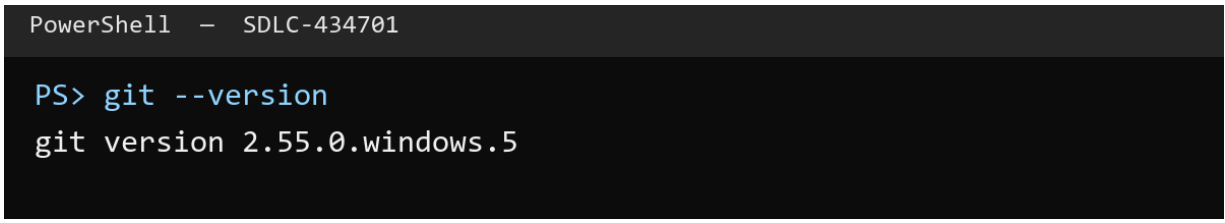


Рисунок 3.1.1 – Проверка версии Git

3.2 Настройка конфигурации Git

Для коммитов в учебном репозитории заданы реальное имя студента с номером группы и адрес GitHub no-reply. Настройка выполнена локально для этого репозитория с помощью команд:

```
git config --local user.name "Юденко Е.А., гр. 434701"
git config --local user.email
"246707085+123GDminer@users.noreply.github.com"
```

Проверка конфигурации, представленная на рисунке 3.2.1, выполнена командами git config --local user.name и git config --local user.email. В выводе отображаются заданные имя и адрес.

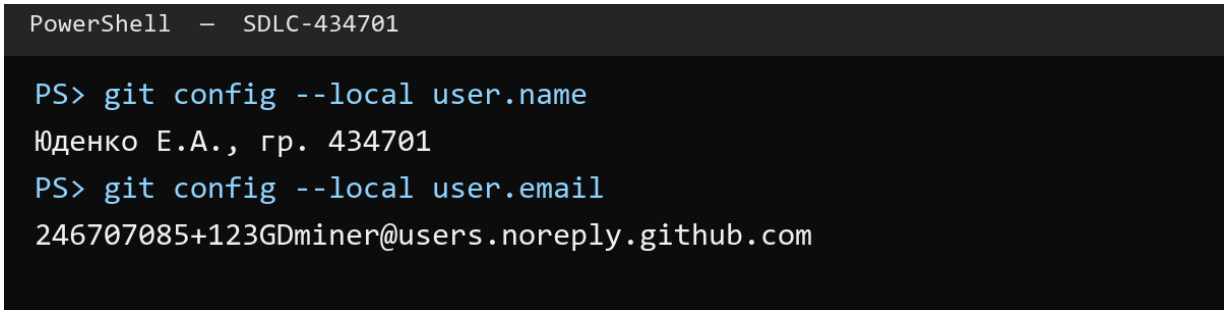


Рисунок 3.2.1 – Проверка конфигурации Git

Для авторизации при работе с удалённым репозиторием создан SSH-ключ ED25519 с помощью команды:

```
ssh-keygen -t ed25519 -C
"246707085+123GDminer@users.noreply.github.com"
```

Ключ сохранён в каталоге C:\Users\MORFIK\.ssh. Парольная фраза не задавалась. Для проверки соединения выполнена команда ssh -T git@github.com; GitHub подтвердил вход под учётной записью 123GDminer.

В результате были созданы два файла:

– id_ed25519 – приватный ключ;

– id_ed25519.pub – публичный ключ.

Публичная часть ключа добавлена в настройки аккаунта GitHub как Authentication Key. Добавленный SSH-ключ представлен на рисунке 3.2.2.

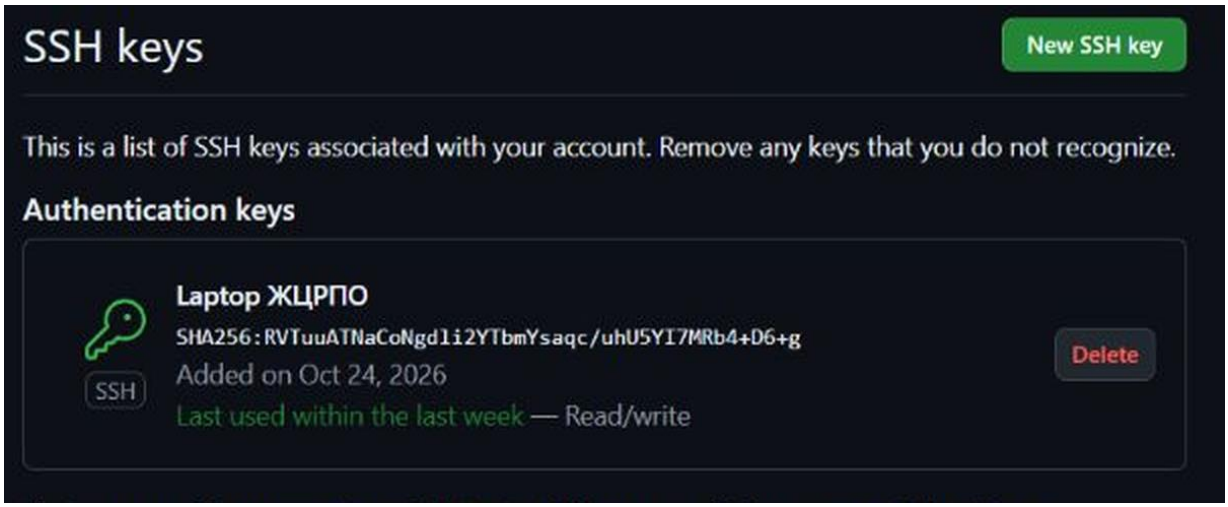


Рисунок 3.2.2 – SSH-ключ в аккаунте GitHub

3.3 Fork репозитория

В соответствии с заданием выполнен fork репозитория BSUIR-SDLC/SDLC. В аккаунте 123GDminer создана открытая личная копия SDLC-434701, представленная на рисунке 3.3.1.

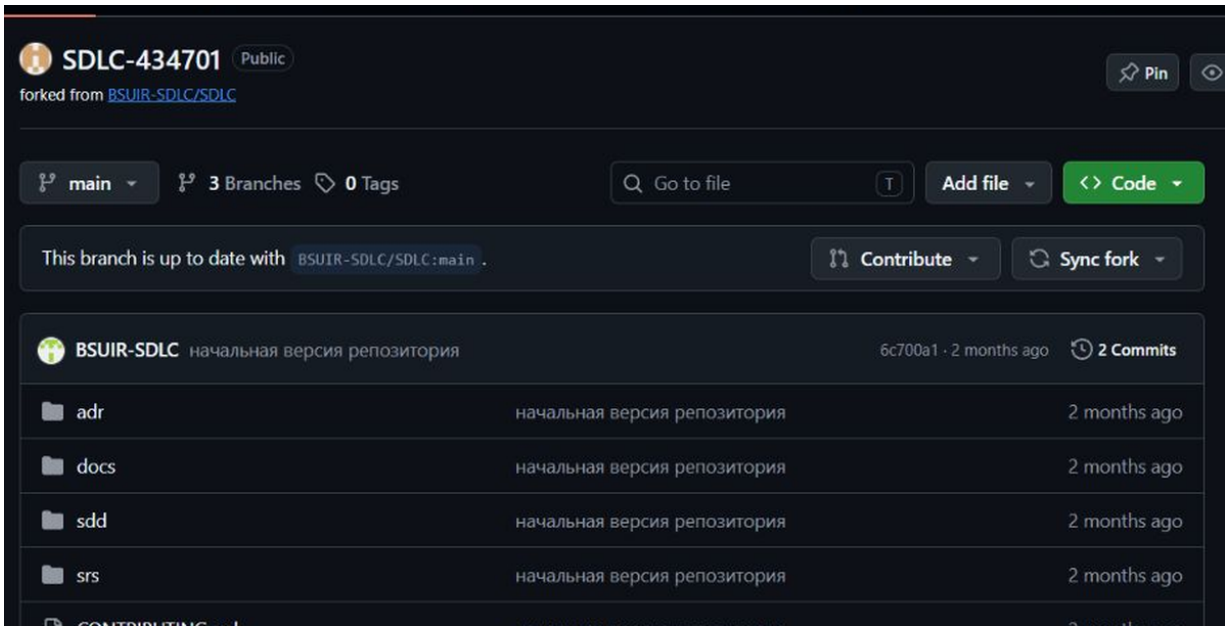


Рисунок 3.3.1 – Fork репозитория

Репозиторий является публичным, что позволяет преподавателю контролировать ход работы. На странице указаны исходный репозиторий, ветка main и три ветки после загрузки изменений. Ссылка на созданный репозиторий: https://github.com/123GDminer/SDLC-434701.

3.4 Выполнение операций с репозиторием в консоли

Операции с Git выполнялись через командную строку.
Локальная копия созданного fork также проверена
клонированием по SSH:

```
git clone git@github.com:123GDminer/SDLC-434701.git
```

Клонирование по SSH выполнено успешно. Изучена базовая структура репозитория: каталоги adr, docs, sdd и srs, а также файлы README.md и LICENSE.md.

Относительно ветки main создана ветка lab1 и выполнен переход в неё:

```
git switch -c lab1
```

В ветку lab1 скопированы исходный код C++, настройки сборки VS Code и отчёт лабораторной работы № 1 в форматах DOCX и PDF.

Изменения в рабочем дереве проанализированы командой:

```
git status
```

До фиксации Git определил добавленные материалы как новые файлы. После commit и push команда git status показала чистое рабочее дерево ветки lab1, что представлено на рисунке 3.4.1.

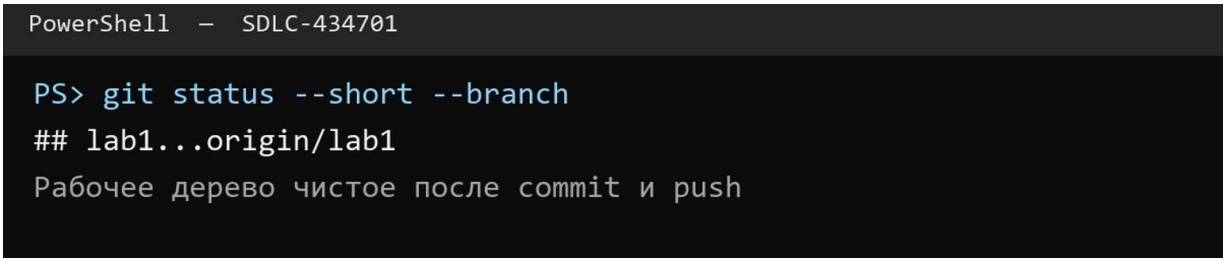


Рисунок 3.4.1 – Состояние рабочего дерева после отправки изменений

В корне проекта создан файл .gitignore, исключающий результаты сборки и отладки MSVC, временные файлы редактора и системы. Исходный код и отчёты оставлены под контролем версий.

Изменения добавлены в область подготовки, зафиксированы и отправлены в удалённую ветку lab1. Содержимое ветки представлено на рисунке 3.4.2.

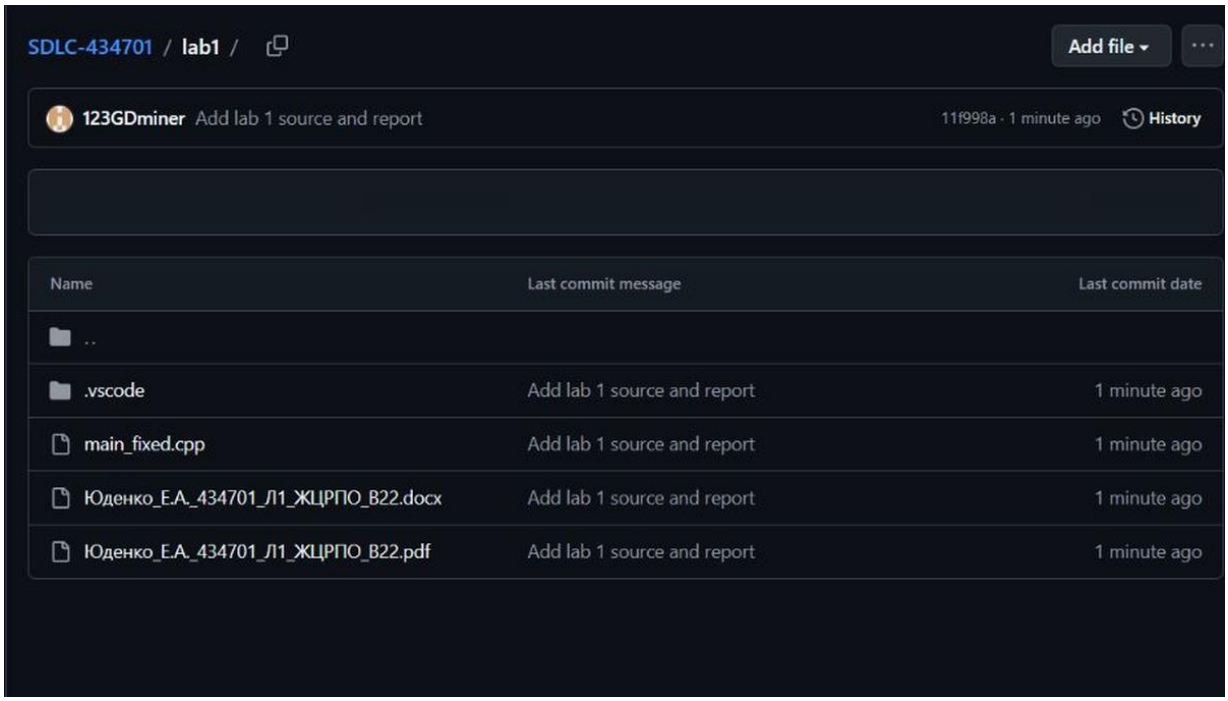


Рисунок 3.4.2 – Ветка lab1 в удаленном репозитории

3.5 Создание ветки ProjectTemplate

После завершения работы в ветке lab1 выполнен возврат в ветку main:

```
git switch main
```

Относительно текущего мастера создана новая ветка ProjectTemplate и выполнен переход в нее:

```
git switch -c ProjectTemplate
```

В корневом README.md указаны автор, дисциплина, назначение приложения «Сколько раз ты сказал „потом“», применяемые C++ и WinAPI, а также структура репозитория.

В каталоге project подготовлена пустая структура C++-проекта для исходного кода, заголовков и ресурсов. Реализация из лабораторной работы № 1 в эту ветку не переносилась.

В корне репозитория настроен файл .gitignore, исключающий временные файлы и результаты сборки проекта.

Изменения зафиксированы и отправлены в удалённую ветку ProjectTemplate, представленную на рисунке 3.5.1.

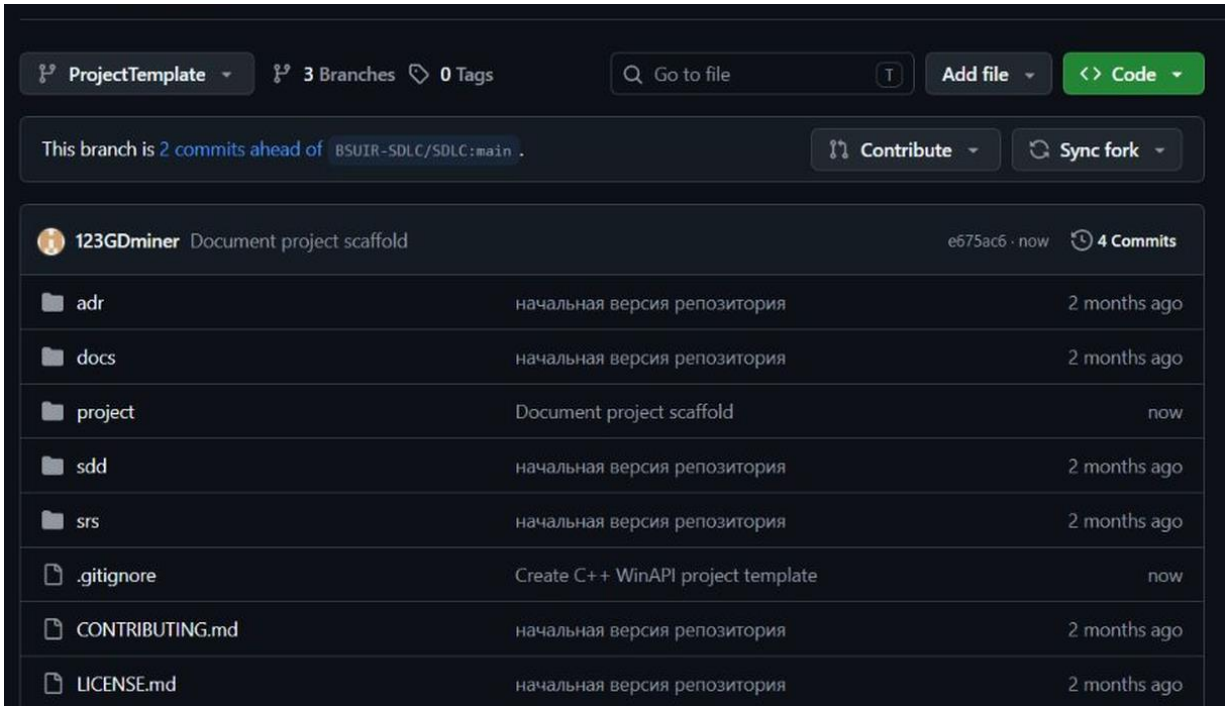


Рисунок 3.5.1 – Ветка ProjectTemplate в удалённом репозитории

4 ОТВЕТЫ НА КОНТРОЛЬНЫЕ ВОПРОСЫ

4.1 В чем преимущество Git перед SVN?

Git является распределенной системой контроля версий, тогда как SVN – централизованной. Главное преимущество Git заключается в том, что практически все операции выполняются локально: коммит, просмотр истории, создание веток и слияние не требуют подключения к серверу. Кроме того, Git обеспечивает легкое и быстрое ветвление, высокую скорость работы и полную локальную копию репозитория с историей, что делает его устойчивым к сбоям центрального сервера.

4.2 В чем преимущество распределенных репозиториев?

В распределенных системах каждый разработчик имеет полную локальную копию репозитория, включая всю историю изменений. Это позволяет работать автономно, без подключения к сети, и синхронизироваться с другими репозиториями только при необходимости. Кроме того, распределенные репозитории обеспечивают устойчивость к потере центрального сервера и поддерживают гибкие модели совместной работы.

4.3 Граф типичной работы с веткой master в Git.

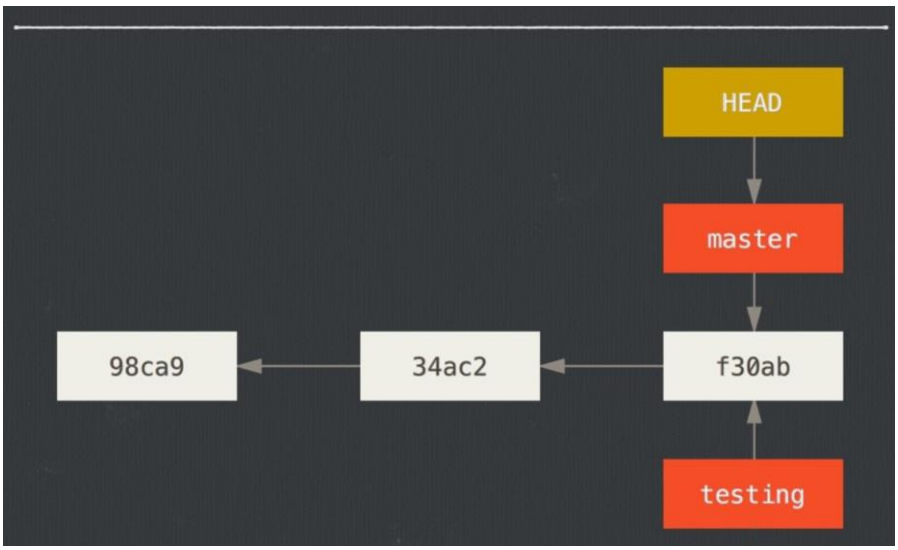


Рисунок 4.3.1 – граф работы с веткой master

4.4 Наиболее часто используемые команды командной строки.

К наиболее часто используемым командам Git относятся: `git clone` – для клонирования удаленного репозитория, `git status` – для просмотра состояния рабочего дерева, `git add` – для добавления изменений в область

подготовки, `git commit` – для фиксации изменений в локальной истории, `git push` – для отправки изменений в удаленный репозиторий, `git pull` и `git fetch` – для получения изменений из удаленного репозитория, `git branch`, `git checkout` и `git merge` – для работы с ветками, а также `git log` – для просмотра истории коммитов.

4.5 Что нужно делать, если кто-то успел внести в удаленный репозиторий изменения?

Если кто-то уже успел внести изменения в удаленный репозиторий, перед отправкой своих изменений необходимо сначала получить эти изменения к себе командой `git pull`.

Эта команда выполняет два действия: загружает изменения из удаленного репозитория и сразу выполняет их слияние с текущей локальной веткой. Если изменения не конфликтуют, Git автоматически выполнит слияние, после чего можно отправить свои изменения командой `git push`.

Если же возник конфликт, Git сообщит о нем – потребуется вручную разрешить конфликтующие файлы, затем добавить их в область подготовки командой `git add`, зафиксировать командой `git commit` и отправить в удаленный репозиторий командой `git push`. В качестве альтернативы можно использовать `git fetch` с последующим `git merge` или `git pull --rebase` для получения линейной истории без лишнего коммита слияния.

Такой подход позволяет избежать потери чужих изменений, которые уже были зафиксированы другими участниками проекта, и корректно синхронизировать локальную и удаленную версии репозитория, обеспечивая согласованность истории коммитов и целостность проекта при совместной работе.

ЗАКЛЮЧЕНИЕ

В ходе лабораторной работы изучена система контроля версий Git, рассмотрены её особенности и основные операции с локальным и удалённым репозиторием. Для учебного репозитория настроены имя студента, номер группы и адрес для коммитов; создан и добавлен в GitHub SSH-ключ.

Выполнен fork предложенного репозитория и проверено его клонирование по SSH. От ветки main создана ветка lab1 с материалами первой лабораторной работы. Настроен .gitignore; изменения зафиксированы и отправлены в удалённый репозиторий.

От ветки main отдельно создана ветка ProjectTemplate. В ней обновлён README.md с описанием проекта, подготовлена пустая структура C++/WinAPI-приложения и настроен .gitignore. Изменения зафиксированы и отправлены в удалённый репозиторий.

Операции с Git выполнялись через командную строку. Состояние веток и содержимое репозитория проверены на GitHub.

Ветки lab1 и ProjectTemplate доступны в открытом репозитории. Слияние ProjectTemplate с main должно выполняться только после одобрения преподавателя и в его присутствии; на момент подготовки отчёта оно не выполнялось.